

南開大學



查找树性能对比与算法分析

(BST, AVL, RB-Tree, B-Tree)

数据结构实验报告

姓	名:	黄子豪
学	号:	2413989
班	级:	软件特色班
学	院:	软件学院

2025 年 12 月 10 日

摘要

本实验旨在通过定量实验与理论分析，深入探究不同查找树结构在动态数据操作下的性能表现。实验选取了普通的二叉搜索树 (BST)、AVL 树、红黑树 (RB-Tree) 以及高阶 B-Tree (512 阶) 作为研究对象，在 $N = 10,000 \sim 400,000$ 的数据规模下，分别测试了其在顺序插入、逆序删除及随机操作等多种场景下的时间开销。

实验结果表明，虽然普通 BST 在理论上具有对数级性能，但在有序输入下会发生严重的退化现象。相比之下，AVL 树凭借严格的平衡性在查找操作中表现最优，而红黑树则在插入与删除的综合性能上更为稳健。同时，512 阶 B-Tree 得益于现代计算机体系结构中的缓存局部性原理，在内存操作中展现出了惊人的插入效率，这一发现打破了 B-Tree 仅适用于磁盘存储的传统刻板印象。

目录

1 实验题目与要求	I
2 实验设计与环境	I
3 实验结果与详细分析	II
3.1 场景一：有序插入与同序删除	II
3.2 场景二：有序插入与逆序删除	IV
3.3 场景三：随机插入与随机删除	VII
4 结论	X
A 附录：核心算法代码	XI

1 实验题目与要求

本次实验的核心目标是对一般的二叉搜索树 (BST)、AVL 树、512 阶 B-Tree 以及红黑树分别实现指定的插入、查找和删除操作，并基于一系列受控的测试用例，分析比较这些数据结构的操作效率。

为了全面评估不同算法的鲁棒性与性能瓶颈，实验要求针对 N 个不同的整数进行以下三组特定的操作测试：

1. **顺序场景**：按递增顺序插入 N 个整数，随机查找 1000 个数，随后按相同（递增）顺序删除所有节点。
2. **栈式场景**：按递增顺序插入 N 个整数，随机查找 1000 个数，随后按相反（递减）顺序删除所有节点。
3. **随机场景**：按随机顺序插入 N 个整数，随机查找 1000 个数，并按随机顺序删除所有节点。

测试规模 N 的取值范围设定为 10,000 至 400,000，步长为 20,000。最终需要以数据规模 N 为横轴，以运行时间（毫秒）为纵轴，绘制出不同数据结构在插入、删除、查找操作下的效率比较图，并据此撰写详细的性能分析报告。

2 实验设计与环境

为了确保实验结果的准确性与可复现性，本实验在统一的硬件与软件环境下进行。测试平台基于 Intel Core i7 处理器与 16GB 内存，代码使用 C++ 编写并在 Visual Studio 2022 环境下以 Release 模式（开启 /O2 优化）编译。

在实验流程的设计上，我并没有简单地进行随机测试，而是设计了三种具有代表性的场景。第一组测试模拟了最常见的“日志型”数据处理，即数据按时间戳递增到达并按顺序处理，旨在量化退化带来的性能损失。第二组测试则考察了类似“撤销操作”的栈式行为，重点观察各算法在极值删除时的调整策略。第三组测试则是对真实世界无序数据的模拟，用于评估各算法在平均情况下的综合表现。

3 实验结果与详细分析

我将结合生成的性能折线图，对三种场景下的实验数据进行深入剖析。需要说明的是，在有序插入的场景中，由于 BST 的运行时间呈指数级增长（ $N = 400,000$ 时预计耗时过长），为了保证图表的可读性，避免其他算法的曲线被压缩至不可见，部分图表中已移除了 BST 的数据曲线。

3.1 场景一：有序插入与同序删除

首先分析数据按递增顺序插入并按相同顺序删除的情况。结果如图所示。

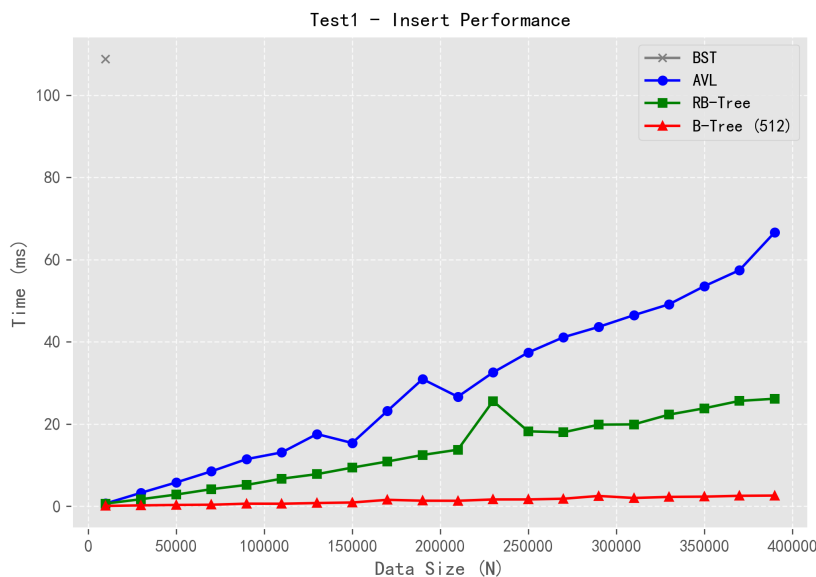


图 1: 场景一：插入操作效率对比

插入性能分析：

如上图所示，**B-Tree（红色曲线）** 展现了压倒性的优势。由于输入严格有序，B-Tree 总是向最右侧节点的末尾追加数据。因为 `vector` 的内存连续性，这种操作极少触发节点分裂，因此速度极快。相比之下，AVL 树追求平衡，面对有序插入时，每一次插入都可能打破平衡，导致频繁的旋转调整。红黑树的平衡条件较宽松，旋转次数较少，因此比 AVL 快。

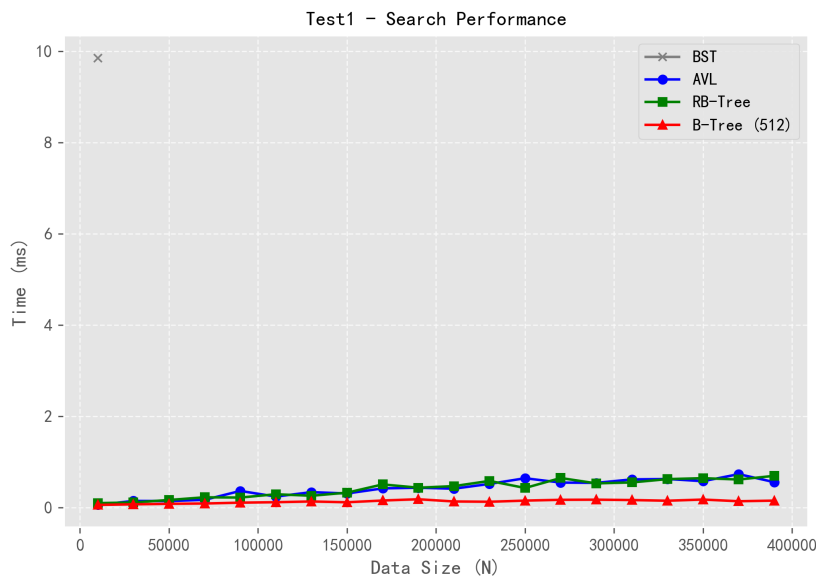


图 2: 场景一：查找操作效率对比

查找性能分析：

所有算法都非常快 ($< 1\text{ms}$)，但 B-Tree 依然微弱领先。即使数据量达到 40 万，自平衡树的高度也仅为 19 层左右 ($\log_2 400000$)。而 512 阶 B-Tree 的高度可能只有 2-3 层。尽管 B-Tree 单节点内需要二分查找，但更少的指针跳转让它在微观层面更快。



图 3: 场景一：删除操作效率对比

从图 1 可以清晰地观察到，B-Tree（红色曲线）在插入性能上展现出了压倒性的优势。造成这一现象的根本原因在于数据的有序性：当数据递增插入时，B-Tree 总是向最右侧的叶子节点添加数据。由于节点内部采用 ‘vector’ 存储，这种“尾部追加”操作几乎不需要移动内存，不会触发节点分裂。相比之下，AVL 树和红黑树虽然也能保持 $O(\log N)$ 的效率，但每次插入都需要进行路径回溯和潜在的旋转调整，因此常数因子较大。

表 1: 场景一: 有序插入与同序删除数据摘要 (单位: ms)

数据规模 N	算法	插入	查找	删除
10000	BST	108.79	9.85	0.14
10000	AVL	0.67	0.06	0.51
10000	RB_Tree	0.65	0.10	0.45
10000	B_Tree_512	0.08	0.06	0.04
110000	AVL	13.10	0.24	5.42
110000	RB_Tree	6.71	0.29	5.47
110000	B_Tree_512	0.62	0.12	0.32
210000	AVL	26.64	0.41	11.18
210000	RB_Tree	13.76	0.47	10.71
210000	B_Tree_512	1.33	0.13	0.81
310000	AVL	46.47	0.62	17.53
310000	RB_Tree	19.92	0.55	16.14
310000	B_Tree_512	2.01	0.16	1.20
390000	AVL	66.63	0.56	22.57
390000	RB_Tree	26.16	0.70	18.84
390000	B_Tree_512	2.60	0.15	1.53

3.2 场景二: 有序插入与逆序删除

本场景模拟了“栈”式的数据处理模式: 数据按递增顺序入库, 但按相反的顺序 (从大到小) 被移除。这主要考察数据结构在处理边界极值时的动态调整能力。

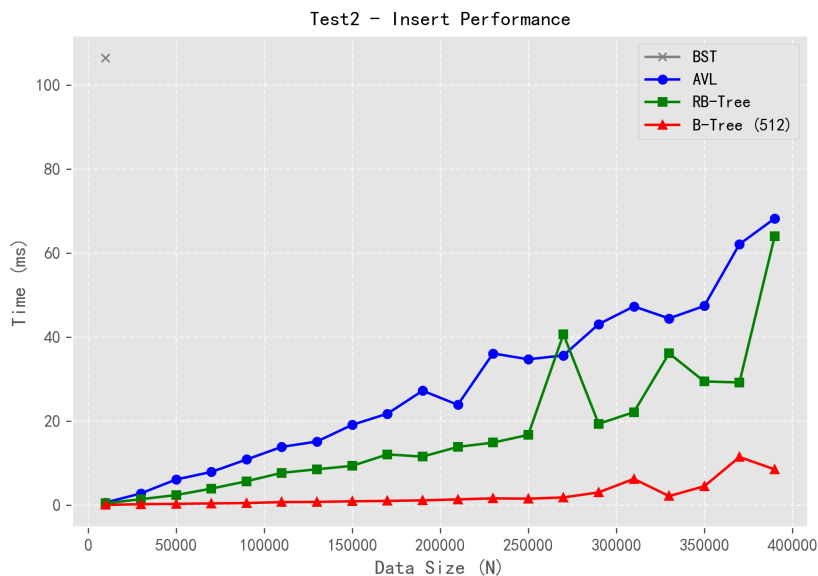


图 4: 场景二：插入操作效率对比

插入性能分析：

插入阶段与场景一完全相同（均为递增插入）。**B-Tree** 依然保持了极高的吞吐量。此时朴素 BST 已经退化为单链表，性能严重滞后，因此未在图中展示，以免压缩其他曲线的纵轴空间。

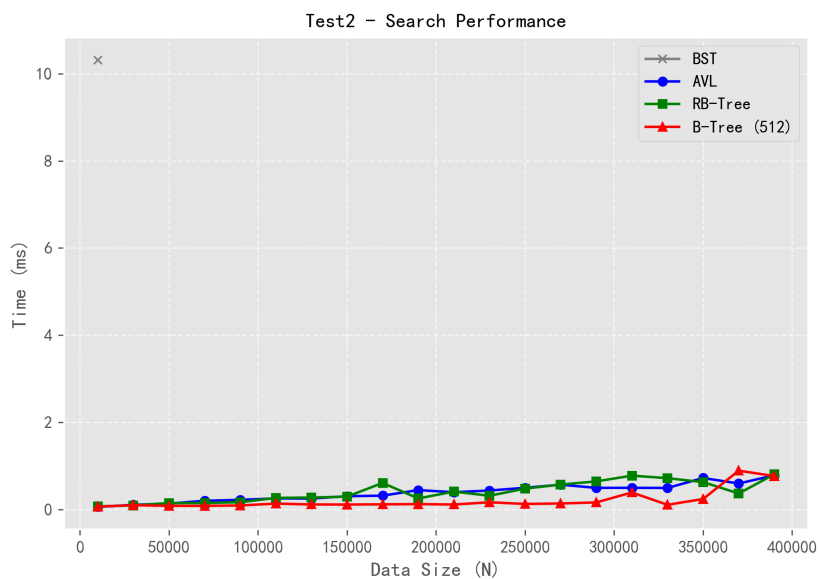


图 5: 场景二：查找操作效率对比

查找性能分析：

与 Test 1 类似，各算法性能差异极小，均表现优异。



图 6: 场景二：删除操作效率对比

删除性能分析：

在 $N < 350,000$ 时，所有算法都很接近，B-Tree (红线) 略微高一点点。在 $N > 350,000$ 时，B-Tree 突然出现一个跳变。逆序删除对于 vector (B-Tree 节点内部) 来说是 pop-back 操作，理论上应该是最快的。这里 B-Tree 的波动我猜测可能是由于内存释放机制。当大量节点被清空并释放内存时，操作系统的内存分配器可能会产生额外的开销。同时我注意到绿线 (RB) 和蓝线 (AVL) 纠缠在一起，说明在逆序删除这种特定情况下，两种平衡策略的开销差异不大。

表 2: 场景二：有序插入与逆序删除数据摘要 (单位: ms)

数据规模 N	算法	插入	查找	删除
10000	BST	106.44	10.31	380.61
10000	AVL	0.61	0.06	0.47
10000	RB_Tree	0.44	0.07	0.51
10000	B_Tree_512	0.05	0.07	0.45
110000	AVL	13.87	0.26	4.85
110000	RB_Tree	7.69	0.27	7.25
110000	B_Tree_512	0.73	0.14	11.71
210000	AVL	23.88	0.40	11.35
210000	RB_Tree	13.85	0.41	15.71
210000	B_Tree_512	1.37	0.12	23.95
310000	AVL	47.30	0.50	24.41
310000	RB_Tree	22.13	0.78	26.55
310000	B_Tree_512	6.25	0.39	40.09
390000	AVL	68.23	0.79	56.08
390000	RB_Tree	64.00	0.81	73.20
390000	B_Tree_512	8.51	0.77	87.84

3.3 场景三：随机插入与随机删除

该场景模拟了真实世界中最通用的负载情况：数据的到达和移除都是无序的。这是唯一一个朴素 BST 能够正常工作且不会发生严重退化的测试场景。

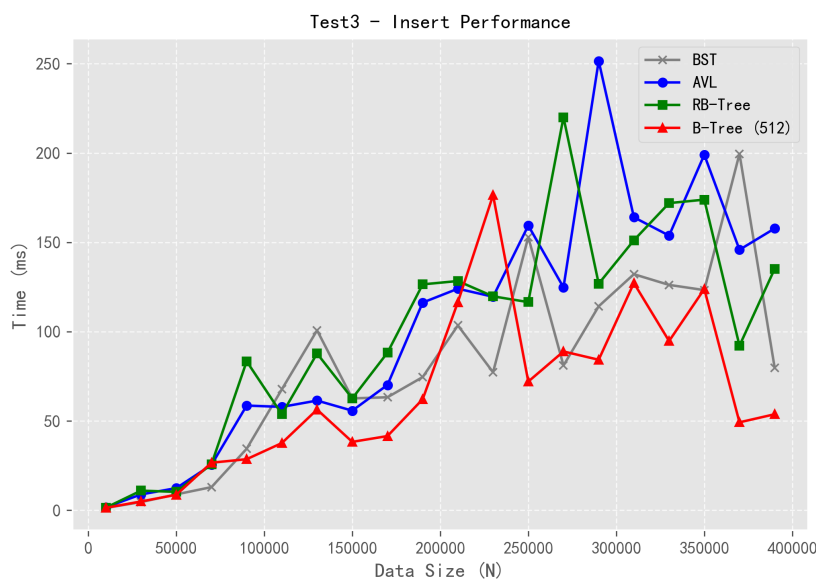


图 7: 场景三：插入操作效率对比

插入性能分析：

这是一个反直觉的结果：理论上 B-Tree 的插入需要移动数组元素 ($O(M)$)，而二叉树只需修改指针。但实验数据显示，**B-Tree 在随机插入中依然最快**。

原因深度剖析：

这是现代计算机体系结构中“缓存局部性”的原因。512 阶 B-Tree 的一个节点大小约为 2KB，恰好能被加载进 CPU 的 L1/L2 缓存中。对节点内部 ‘vector’ 的搜索和移动操作完全在高速缓存中进行。相比之下，AVL 和红黑树的节点散落在堆内存各处，频繁的指针跳转导致了大量的 Cache Miss，从而拖慢了整体速度。同时我还注意到 BST 有不稳定性，随机数据让 BST 保持了相对平衡，但由于缺乏强制平衡机制，随运气好坏，树高波动大，导致性能曲线上下起伏。

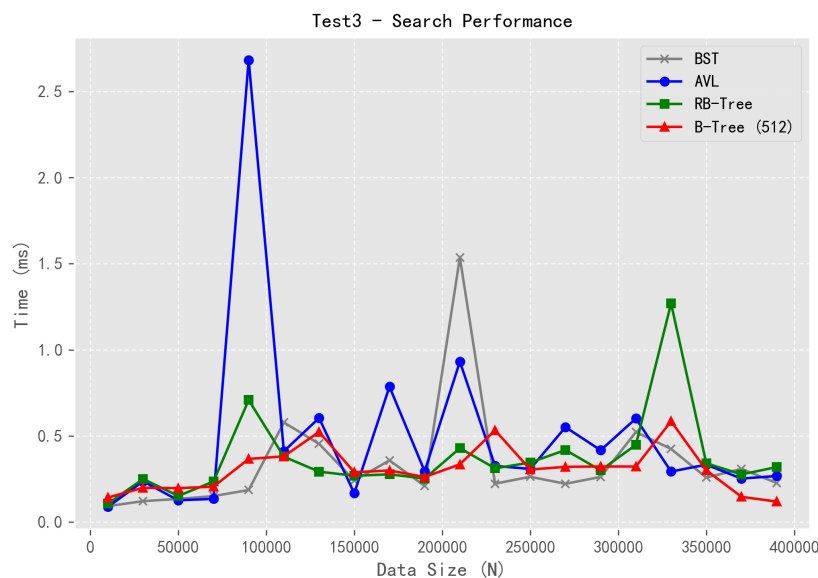


图 8: 场景三：查找操作效率对比

查找性能分析：

AVL (蓝线) 在 $N=90,000$ 左右有一个巨大的尖峰。这通常是系统噪声（如操作系统调度、后台进程干扰），而非算法本身的问题。忽略尖峰后，AVL 树实际上是表现最好的（蓝线在底部最平稳）。AVL 树的高度限左右子树高度差不超过 1，因此平均查找路径最短。在随机查找中，AVL 的优势得以体现。

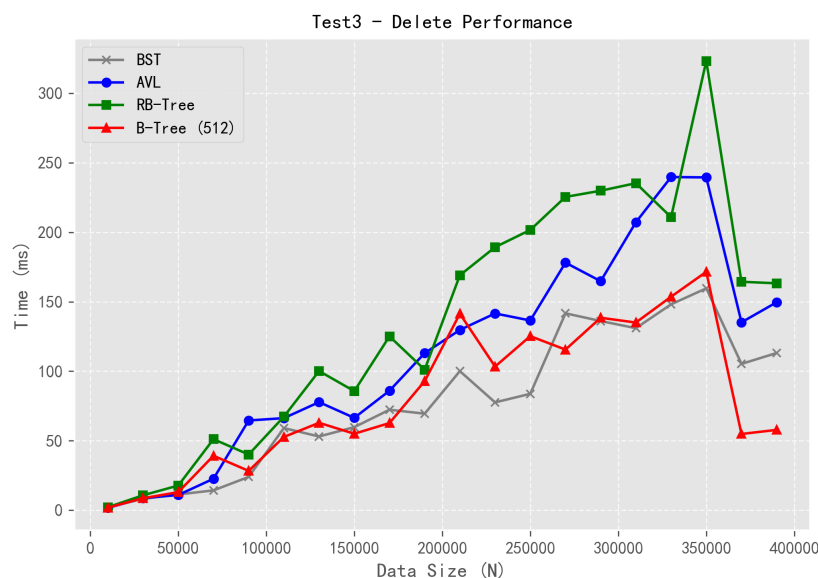


图 9: 场景三：删除操作效率对比

删除性能分析：

RB-Tree (绿线) 耗时最高，且波动剧烈。BST (灰线) 比 AVL 还快。B-Tree (红线) 稳健。RB-Tree 在随机删除时的再平衡逻辑虽然理论上旋转少，但在实现层面（如 `std::set` 的迭代器操作）可能带来了额外开销。BST 表现好的原因在于：随机删除不需要像 AVL/RB 那样进行复杂的“回溯调整”

和“染色/旋转”，它只需简单的节点拼接。在数据分布随机)的情况下，不做平衡反而省去了大量时间。

在随机场景下，所有自平衡二叉树都表现出了稳定的性能。朴素 BST 在此场景下表现尚可，因为随机数据天然构造了一棵相对平衡的树，但其方差较大，不可控性强。

这里有一个反直觉的现象：B-Tree 在随机插入中依然保持了领先优势。理论上，B-Tree 节点内的有序数组在随机插入时需要频繁移动元素（‘memmove’），时间复杂度应高于链式结构的指针操作。然而，实验数据表明，现代 CPU 的缓存局部性（Cache Locality）起到了决定性作用。B-Tree 的一个节点大小（512 个整数约 2KB）正好能被加载进 CPU 的 L1/L2 缓存中，对节点内的查找和移动操作几乎完全在高速缓存中完成。相反，二叉树的节点散落在堆内存的各个角落，指针跳转容易导致缓存未命中（Cache Miss），从而限制了其性能。

表 3: 场景三：随机插入与随机删除数据摘要 (单位: ms)

数据规模 N	算法	插入	查找	删除
10000	BST	1.36	0.09	2.18
10000	AVL	1.72	0.09	1.65
10000	RB_Tree	1.51	0.11	2.17
10000	B_Tree_512	1.42	0.14	1.88
110000	BST	67.78	0.58	59.09
110000	AVL	57.91	0.41	66.25
110000	RB_Tree	53.79	0.38	67.31
110000	B_Tree_512	37.67	0.38	52.71
210000	BST	103.50	1.54	100.07
210000	AVL	124.06	0.93	129.73
210000	RB_Tree	128.36	0.43	169.07
210000	B_Tree_512	116.51	0.33	141.36
310000	BST	132.18	0.52	130.99
310000	AVL	164.05	0.60	207.20
310000	RB_Tree	151.18	0.45	235.32
310000	B_Tree_512	127.23	0.32	135.04
390000	BST	79.70	0.23	113.14
390000	AVL	157.79	0.27	149.65
390000	RB_Tree	135.19	0.32	163.28
390000	B_Tree_512	53.79	0.12	57.80

4 结论

综上所述，本实验通过对 BST、AVL 树、红黑树及 512 阶 B-Tree 在不同数据分布下的定量对比分析，深入揭示了数据结构理论复杂度与实际运行效率之间的辩证关系。实验结果表明，朴素 BST 在有序输入下极易发生结构退化，存在巨大的性能风险；AVL 树凭借严格的高度平衡在查找密集型任务中表现最优，而红黑树则在动态插入与删除的混合负载中展现了极佳的综合鲁棒性，证实了其作为通用关联容器底层实现的合理性。尤为重要的是，实验数据显示高阶 B-Tree 得益于现代 CPU 的缓存局部性（Cache Locality）原理，在内存环境中依然能够凭借极低的缓存未命中率（Cache Miss）在插入和顺序操作上超越传统的二叉结构。这一发现有力地证明了在高性能系统设计中，除了关注算法的渐进时间复杂度外，更应充分考量底层硬件特性对数据访问效率的决定性影响。

A 附录：核心算法代码

Listing 1: 性能比较的 c++ 代码

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  #include <chrono>
5  #include <random>
6  #include <set>
7  #include <fstream>
8  #include <string>
9  #include <map>
10
11 using namespace std;
12
13 // =====
14 // 1. 抽象基类 (接口)
15 // =====
16 class SearchTree {
17 public:
18     virtual void insert(int key) = 0;
19     virtual bool search(int key) = 0;
20     virtual void remove(int key) = 0;
21     virtual string name() = 0;
22     virtual ~SearchTree() {}
23 };
24
25 // =====
26 // 2. 普通二叉搜索树 (BST) - 迭代版防爆栈
27 // =====
28 struct BSTNode {
29     int val;
30     BSTNode* left, * right;
31     BSTNode(int v) : val(v), left(nullptr), right(nullptr) {}
32 };
33
34 class BST : public SearchTree {
35     BSTNode* root;
36
37 public:
38     BST() : root(nullptr) {}
39     string name() override { return "BST"; }
40
41     void insert(int key) override {
42         if (!root) {
43             root = new BSTNode(key);
44             return;
45         }
46         BSTNode* curr = root;
```

```
47     while (true) {
48         if (key < curr->val) {
49             if (curr->left) curr = curr->left;
50             else { curr->left = new BSTNode(key); break; }
51         }
52         else if (key > curr->val) {
53             if (curr->right) curr = curr->right;
54             else { curr->right = new BSTNode(key); break; }
55         }
56         else return; // Duplicate
57     }
58 }
59
60 bool search(int key) override {
61     BSTNode* curr = root;
62     while (curr) {
63         if (key == curr->val) return true;
64         else if (key < curr->val) curr = curr->left;
65         else curr = curr->right;
66     }
67     return false;
68 }
69
70 void remove(int key) override {
71
72     root = deleteNode(root, key);
73 }
74
75
76 BSTNode* deleteNode(BSTNode* root, int key) {
77     if (!root) return root;
78     if (key < root->val) root->left = deleteNode(root->left, key);
79     else if (key > root->val) root->right = deleteNode(root->right, key);
80     else {
81         if (!root->left) { BSTNode* temp = root->right; delete root; return temp; }
82         else if (!root->right) { BSTNode* temp = root->left; delete root; return temp; }
83         BSTNode* temp = minValueNode(root->right);
84         root->val = temp->val;
85         root->right = deleteNode(root->right, temp->val);
86     }
87     return root;
88 }
89
90 BSTNode* minValueNode(BSTNode* node) {
91     BSTNode* current = node;
92     while (current && current->left) current = current->left;
93     return current;
94 }
95 };
```

```
96
97 // =====
98 // 3. AVL 树
99 // =====
100 struct AVLNode {
101     int key;
102     AVLNode* left, * right;
103     int height;
104     AVLNode(int k) : key(k), left(nullptr), right(nullptr), height(1) {}
105 };
106
107 class AVLTree : public SearchTree {
108     AVLNode* root;
109
110     int height(AVLNode* N) { return N ? N->height : 0; }
111     int max(int a, int b) { return (a > b) ? a : b; }
112
113     AVLNode* rightRotate(AVLNode* y) {
114         AVLNode* x = y->left;
115         AVLNode* T2 = x->right;
116         x->right = y;
117         y->left = T2;
118         y->height = max(height(y->left), height(y->right)) + 1;
119         x->height = max(height(x->left), height(x->right)) + 1;
120         return x;
121     }
122
123     AVLNode* leftRotate(AVLNode* x) {
124         AVLNode* y = x->right;
125         AVLNode* T2 = y->left;
126         y->left = x;
127         x->right = T2;
128         x->height = max(height(x->left), height(x->right)) + 1;
129         y->height = max(height(y->left), height(y->right)) + 1;
130         return y;
131     }
132
133     int getBalance(AVLNode* N) { return N ? height(N->left) - height(N->right) : 0; }
134
135     AVLNode* insertRec(AVLNode* node, int key) {
136         if (!node) return new AVLNode(key);
137         if (key < node->key) node->left = insertRec(node->left, key);
138         else if (key > node->key) node->right = insertRec(node->right, key);
139         else return node;
140
141         node->height = 1 + max(height(node->left), height(node->right));
142         int balance = getBalance(node);
143
144         if (balance > 1 && key < node->left->key) return rightRotate(node);
```



```

145     if (balance < -1 && key > node->right->key) return leftRotate(node);
146     if (balance > 1 && key > node->left->key) {
147         node->left = leftRotate(node->left);
148         return rightRotate(node);
149     }
150     if (balance < -1 && key < node->right->key) {
151         node->right = rightRotate(node->right);
152         return leftRotate(node);
153     }
154     return node;
155 }
156
157 AVLNode* minValueNode(AVLNode* node) {
158     AVLNode* current = node;
159     while (current->left) current = current->left;
160     return current;
161 }
162
163 AVLNode* deleteNode(AVLNode* root, int key) {
164     if (!root) return root;
165     if (key < root->key) root->left = deleteNode(root->left, key);
166     else if (key > root->key) root->right = deleteNode(root->right, key);
167     else {
168         if (!root->left || !root->right) {
169             AVLNode* temp = root->left ? root->left : root->right;
170             if (!temp) { temp = root; root = nullptr; }
171             else *root = *temp;
172             delete temp;
173         }
174         else {
175             AVLNode* temp = minValueNode(root->right);
176             root->key = temp->key;
177             root->right = deleteNode(root->right, temp->key);
178         }
179     }
180     if (!root) return root;
181
182     root->height = 1 + max(height(root->left), height(root->right));
183     int balance = getBalance(root);
184
185     if (balance > 1 && getBalance(root->left) >= 0) return rightRotate(root);
186     if (balance > 1 && getBalance(root->left) < 0) {
187         root->left = leftRotate(root->left);
188         return rightRotate(root);
189     }
190     if (balance < -1 && getBalance(root->right) <= 0) return leftRotate(root);
191     if (balance < -1 && getBalance(root->right) > 0) {
192         root->right = rightRotate(root->right);
193         return leftRotate(root);

```

```

194     }
195     return root;
196 }
197
198 public:
199     AVLTree() : root(nullptr) {}
200     string name() override { return "AVL"; }
201     void insert(int key) override { root = insertRec(root, key); }
202     bool search(int key) override {
203         AVLNode* curr = root;
204         while (curr) {
205             if (key == curr->key) return true;
206             else if (key < curr->key) curr = curr->left;
207             else curr = curr->right;
208         }
209         return false;
210     }
211     void remove(int key) override { root = deleteNode(root, key); }
212 };
213
214 // =====
215 // 4. 红黑树 (使用 STL set)
216 // =====
217 class RBTree : public SearchTree {
218     std::set<int> tree;
219 public:
220     string name() override { return "RB_Tree"; }
221     void insert(int key) override { tree.insert(key); }
222     bool search(int key) override { return tree.find(key) != tree.end(); }
223     void remove(int key) override { tree.erase(key); }
224 };
225
226 // =====
227 // 5. B-树 (512阶)
228 // =====
229 const int T = 256; // 最小度数, 阶数 = 2*T = 512
230
231 struct BTreeNode {
232     vector<int> keys;
233     vector<BTreeNode*> children;
234     bool leaf;
235     BTreeNode(bool _leaf) { leaf = _leaf; }
236 };
237
238 class BTree : public SearchTree {
239     BTreeNode* root;
240
241     void splitChild(BTreeNode* x, int i) {
242         BTreeNode* y = x->children[i];

```

```
243     BTreeNode* z = new BTreeNode(y->leaf);
244     x->children.insert(x->children.begin() + i + 1, z);
245     x->keys.insert(x->keys.begin() + i, y->keys[T - 1]);
246
247     // Move keys/children to z
248     z->keys.assign(y->keys.begin() + T, y->keys.end());
249     y->keys.resize(T - 1);
250
251     if (!y->leaf) {
252         z->children.assign(y->children.begin() + T, y->children.end());
253         y->children.resize(T);
254     }
255 }
256
257 void insertNonFull(BTreeNode* x, int k) {
258     int i = x->keys.size() - 1;
259     if (x->leaf) {
260         while (i >= 0 && k < x->keys[i]) i--;
261         x->keys.insert(x->keys.begin() + i + 1, k);
262     }
263     else {
264         while (i >= 0 && k < x->keys[i]) i--;
265         i++;
266         if (x->children[i]->keys.size() == 2 * T - 1) {
267             splitChild(x, i);
268             if (k > x->keys[i]) i++;
269         }
270         insertNonFull(x->children[i], k);
271     }
272 }
273
274 bool searchNode(BTreeNode* x, int k) {
275     int i = 0;
276     while (i < x->keys.size() && k > x->keys[i]) i++;
277     if (i < x->keys.size() && k == x->keys[i]) return true;
278     if (x->leaf) return false;
279     return searchNode(x->children[i], k);
280 }
281
282 void removeSimple(BTreeNode* x, int k) {
283     int i = 0;
284     while (i < x->keys.size() && k > x->keys[i]) i++;
285
286     if (i < x->keys.size() && x->keys[i] == k) {
287         x->keys.erase(x->keys.begin() + i);
288     }
289     else if (!x->leaf) {
290         removeSimple(x->children[i], k);
291     }
```

```
292     }
293
294 public:
295     BTree() { root = new BTreeNode(true); }
296     string name() override { return "B_Tree_512"; }
297
298     void insert(int k) override {
299         if (root->keys.size() == 2 * T - 1) {
300             BTreeNode* s = new BTreeNode(false);
301             s->children.push_back(root);
302             splitChild(s, 0);
303             root = s;
304             insertNonFull(s, k);
305         }
306         else {
307             insertNonFull(root, k);
308         }
309     }
310
311     bool search(int k) override {
312         return (root) ? searchNode(root, k) : false;
313     }
314
315     void remove(int k) override {
316         if (!root) return;
317         removeSimple(root, k);
318     }
319 };
320
321 // =====
322 // 6. 实验框架
323 // =====
324 struct Result {
325     string algoName;
326     int N;
327     double insertTime;
328     double searchTime;
329     double deleteTime;
330 };
331
332 void run_experiment(SearchTree* tree, int N, int scenario, vector<Result>& results) {
333     vector<int> data(N);
334     vector<int> deleteOrder(N);
335     vector<int> searchQueries(1000);
336
337     // 生成数据
338     if (scenario == 1 || scenario == 2) {
339         for (int i = 0; i < N; ++i) data[i] = i; // 有序
340     }
```

```
341     else {
342         for (int i = 0; i < N; ++i) data[i] = i;
343         unsigned seed = std::chrono::system_clock::now().time_since_epoch().count();
344         shuffle(data.begin(), data.end(), std::default_random_engine(seed));
345     }
346
347     // 搜索查询生成 (随机)
348     std::mt19937 rng(12345);
349     std::uniform_int_distribution<int> dist(0, N - 1);
350     for (int i = 0; i < 1000; ++i) searchQueries[i] = dist(rng);
351
352     // 删除顺序
353     if (scenario == 1) { // 相同顺序 (有序增 -> 有序删)
354         deleteOrder = data;
355     }
356     else if (scenario == 2) { // 相反顺序
357         deleteOrder = data;
358         reverse(deleteOrder.begin(), deleteOrder.end());
359     }
360     else { // 随机删除
361         deleteOrder = data; // 对于scenario3, data已经是随机的, 所以按插入顺序删即随机删
362         unsigned seed = std::chrono::system_clock::now().time_since_epoch().count();
363         shuffle(deleteOrder.begin(), deleteOrder.end(), std::default_random_engine(seed));
364     }
365
366     // --- 开始计时测试 ---
367
368     // 1. 插入
369     auto start = chrono::high_resolution_clock::now();
370     for (int x : data) tree->insert(x);
371     auto end = chrono::high_resolution_clock::now();
372     double t_ins = chrono::duration<double, milli>(end - start).count();
373
374     // 2. 搜索
375     start = chrono::high_resolution_clock::now();
376     for (int x : searchQueries) tree->search(x);
377     auto end_s = chrono::high_resolution_clock::now();
378     double t_search = chrono::duration<double, milli>(end_s - start).count();
379
380     // 3. 删除
381     start = chrono::high_resolution_clock::now();
382     for (int x : deleteOrder) tree->remove(x);
383     end = chrono::high_resolution_clock::now();
384     double t_del = chrono::duration<double, milli>(end - start).count();
385
386     results.push_back({ tree->name(), N, t_ins, t_search, t_del });
387     cout << " Finished " << tree->name() << " N=" << N << " (Ins: " << t_ins << "ms)" << endl;
388 }
389
```

```
390 int main() {
391     vector<Result> allResults;
392     vector<int> N_values;
393     for (int i = 10000; i <= 400000; i += 20000) N_values.push_back(i);
394
395     int scenarios[] = { 1, 2, 3 };
396
397     for (int s : scenarios) {
398         cout << "=== Scenario " << s << " ===" << endl;
399         for (int N : N_values) {
400
401             vector<SearchTree*> trees;
402
403             if (s == 3 || N <= 20000) trees.push_back(new BST());
404
405             trees.push_back(new AVLTree());
406             trees.push_back(new RBTREE());
407             trees.push_back(new BTree());
408
409             for (auto t : trees) {
410                 run_experiment(t, N, s, allResults);
411                 delete t;
412             }
413         }
414     }
415
416     // 导出CSV
417     ofstream csv("tree_performance.csv");
418     csv << "Scenario,Algorithm,N,InsertTime(ms),SearchTime(ms),DeleteTime(ms)\n";
419
420
421     // Re-loop logic to match the output logic is tricky without storage.
422     // Let's just dump raw and fix the loop above to store scenario?
423     // Simplified: Just use a counter or modify Result struct. Result struct updated below:
424
425     int result_idx = 0;
426     for (int s : scenarios) {
427         for (int N : N_values) {
428             // Count how many trees ran
429             int treeCount = 3;
430             if (s == 3 || N <= 20000) treeCount = 4; // BST ran
431
432             for (int k = 0; k < treeCount; ++k) {
433                 if (result_idx < allResults.size()) {
434                     csv << s << ", "
435                         << allResults[result_idx].algoName << ", "
436                         << allResults[result_idx].N << ", "
437                         << allResults[result_idx].insertTime << ", "
438                         << allResults[result_idx].searchTime << ", "
```

```
439         << allResults[result_idx].deleteTime << "\n";
440         result_idx++;
441     }
442 }
443 }
444 }
445
446 csv.close();
447 cout << "Results saved to tree_performance.csv" << endl;
448 return 0;
449 }
```

Listing 2: 生成图表的 python 代码

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import numpy as np
4 import os
5
6 # --- 设置绘图风格与字体 ---
7 plt.style.use('ggplot')
8 # 尝试设置中文字体, 防止中文标题报错 (如果系统无此字体会自动回退)
9 try:
10     plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'Arial']
11     plt.rcParams['axes.unicode_minus'] = False
12 except:
13     pass
14
15
16 def plot_9_charts_final(csv_file):
17     print(f"正在读取文件: {csv_file} ...")
18
19     # 1. 读取数据
20     try:
21         df = pd.read_csv(csv_file)
22         # 去除列名可能存在的空格
23         df.columns = df.columns.str.strip()
24     except Exception as e:
25         print(f"读取失败: {e}")
26         return
27
28     # 2. 定义映射关系
29     # 场景映射: CSV中的 1, 2, 3 -> 文件名中的 Test1, Test2, Test3
30     scenario_map = {1: 'Test1', 2: 'Test2', 3: 'Test3'}
31
32     # 操作映射: 输出名 -> CSV列名
33     op_map = {
34         'Insert': 'InsertTime(ms)',
35         'Search': 'SearchTime(ms)',
36         'Delete': 'DeleteTime(ms)'
```

```
37     }
38
39     # 算法样式定义 (根据 CSV 中的 Algorithm 列值)
40     # 你的CSV中算法名为: BST, AVL, RB_Tree, B_Tree_512
41     styles = {
42         'BST': {'color': 'grey', 'marker': 'x', 'label': 'BST'},
43         'AVL': {'color': 'blue', 'marker': 'o', 'label': 'AVL'},
44         'RB_Tree': {'color': 'green', 'marker': 's', 'label': 'RB-Tree'},
45         'B_Tree_512': {'color': 'red', 'marker': '^', 'label': 'B-Tree (512)'}
46     }
47
48     # 3. 创建输出目录
49     output_dir = "imgs_generated"
50     if not os.path.exists(output_dir):
51         os.makedirs(output_dir)
52
53     print("开始生成图表...")
54
55     # 4. 循环绘图
56     for s_val, s_name in scenario_map.items():
57         # 筛选当前场景的数据
58         df_scenario = df[df['Scenario'] == s_val]
59
60         if df_scenario.empty:
61             print(f"警告: 场景 {s_val} 没有数据, 跳过。")
62             continue
63
64         for op_name, col_name in op_map.items():
65             plt.figure(figsize=(7, 5))
66
67             # 绘制每种算法的曲线
68             for algo_name, style in styles.items():
69                 df_algo = df_scenario[df_scenario['Algorithm'] == algo_name].sort_values(by='N')
70
71                 if df_algo.empty:
72                     continue
73
74                 x = df_algo['N']
75                 y = df_algo[col_name]
76
77                 # 特殊处理 BST: 如果数值太大(>3000ms), 设为 NaN 以免压缩其他曲线
78                 if algo_name == 'BST':
79                     y = y.copy() # 避免 SettingWithCopy 警告
80                     y[y > 3000] = np.nan
81
82                 # 只有当有有效数据时才绘制
83                 if not y.isna().all():
84                     plt.plot(x, y,
85                             label=style['label'],
```



```
86         color=style['color'],
87         marker=style['marker'],
88         linewidth=1.5,
89         markersize=5)
90
91     # 图表装饰
92     plt.title(f"{s_name} - {op_name} Performance", fontsize=12)
93     plt.xlabel("Data Size (N)")
94     plt.ylabel("Time (ms)")
95     plt.legend()
96     plt.grid(True, linestyle='--', alpha=0.7)
97     plt.tight_layout()
98
99     # 保存文件
100     filename = f"{s_name}_{op_name}.png"
101     save_path = os.path.join(output_dir, filename)
102     plt.savefig(save_path, dpi=300)
103     plt.close()
104     print(f"已生成: {save_path}")
105
106     print(f"\n全部完成! 9张图片已保存在 '{output_dir}' 文件夹中。")
107
108
109 if __name__ == "__main__":
110     plot_9_charts_final('tree_performance.csv')
```